SECTION A -ASGMT 04 - TREE SEARCH MP

ASSIGNMENT 04 - ZERO AI LAB VERSION

If you are in **Section A**, then for assignment 04 you are using the zero AI lab. All of this will be completed on the laptop computer given to you when you attend any of the hours below. A TA will be there to hand out a computer and answer any questions. Don't forget to bring your robot!

ZeroAI Lab Location:

- MetroTech 6, Room 339.

slots available:

- Thursdays, 6:00pm - 9:00pm
- Fridays, 3:00pm - 6:00pm

ASSIGNMENT 04 GOALS

The goal of assignment 04 is to implement an A* path planner for optimial, collision free navigation of your differential drive robot. This will include the following sub goals:

1. A* for shortest path planning in simulation
2. A* for minimum time path planning in simulation
3. A* planners on your robot hardware

KEY DELIVERABLES

To be submitted on brightspace on the LECTURE section A:

1. Assignment 04 report
2. Video of A* planning executed in simulation
3. Video of A* planning executed on hardware
4. Copy of `lab_4.py` file
5. Copy of `gui_4.py` file

0. REVIEW THE LECTURE

Review the lecture slide deck on A* planning to remember how A* works. You can bring any notes to the Zero AI lab, including notes that came from GenAI toos. However, **NO CODE SHOULD BE IN YOUR NOTES.**

GenAI Tool Guidance: Ask your gen AI tools any clarifying questions on how A* works, why it is optimal, and what the admissible heuristic are.

1. LOCATE AND UPDATE LAB 4 CODE

Locate the `gui_4.py`, `lab_4.py`, and `map_4.py` files on the github drive. They should be on the Assignment 04 folder on the desktop.

Note three key differences with the new assignment 04 GUI:

1. The grid overlaid on the environment. This grid represents the space that your robot is planning within. It is a grid of dimensions 200 x 200, where each cell is 1cm x 1cm in real life.
2. Simulated obstacles can be added to the map via the file. `map_4.py` which you can edit. This file contains a 200x200 array of 1s and 0s where 0s are obstacle free and 1s are obstacles.
3. The A* path *plan button, track plan button* and *text field*, in place of the point tracking button and field. This allows you to enter a position in meters. When the *plan button* is pressed, the A* planner should construct a path to the goal that is visualized on the grid. The *track plan button* calls your point tracker to continually track cells along the planned path.

2. CODE THE A* PLANNER

2. CODE THE A* PLANNER

In the `lab_4.py` file, you will edit the function `A_star_planner()` that inputs the initial position as a list `X_0 = [x_0, y_0]`, the desired final position as a list `X_d = [x_d, y_0d]`, and outputs a list of positions that represents the path `P = [X_0, X_1, X_2, ..., X_d]`. To do this follow the steps below. Add any helper functions you want along the way.

Step 1: Create a node class for your tree search. The node should have the following member variables:

- `X` - a list with elements x, y (i.e. X = [x,y]) representing the node's position in the grid
- `parent` - which contains the parent node
- `g_value` - which represents path cost from the root of the tree to the node
- `h_value` - which represents the estimated path cost from the node to the goal. Make sure this is an admissable heuristic.
- `f_value` - the sum of the g and h values.

Step 2: Create a function `get_neighbors(n)` that inputs a node `n` and returns a list of up to 4 nodes that the function will instantiate. These 4 nodes should those in positions 1 cell in the positive X direction, 1 cell in the negative X direction, 1 cell in the positive Y direction, and 1 cell in the negative Y direction with respect to node `n` in the grid map. Reasons any of these 4 neighboring nodes would not be created and returned include:

- **Out of bounds** - the new node's position would be off the grid map.
- **In collision** - the new node would be located in a position on the map that contains an obstacle according to the `map_4.py` file.
- **Already Added** - the new node has already been created and added to the tree earlier in the search. Including this check stops the planner from searching through cycles in the graph. A simple way to accomplish this is to create a

200x200 array of zeros and change an array value to 1 any time a new node is created and added to a neighbor list.

Step 3: Create a function `is_goal(n, X_des)` that checks if node `n` is located at the goal.

Step 4: Write the function `A_star_planner(X_0, X_d)`. Recall that it inputs the initial position as a list `X_0 = [x_0, y_0]`, the desired final position as a list `X_d = [x_d, y_0d]`, and outputs a list of positions that represents the path `P = [X_0, X_1, X_2, ..., X_d]`.

See the pseudo code in the lecture notes. Your function should create a node at the root of the tree, maintain a list of nodes on the fringe, and return a path once the goal is found. Use the functions from steps 1, 2, and 3 to support this function.

Step 5: Write 3 different unit tests that output the optimal path for the map file provided. Make sure it passes these unit tests. One example unit test is provided here at this link.

3. A* FOR MINIMUM TIME

The shortest path is not always the fastest. Some regions of a room can only be crossed slowly, such as carpet, a crowded area or rough ground. In this section you change the cost function of your A* planner so it finds the minimum time path instead of the shortest one.

Step 1: The traversability map. In `map_4.py`, each map can have "slow_zones". A slow zone is a rectangle `(x_min, y_min, x_max, y_max, k)` in meters. Its traversability k is between 0 and 1, and the robot may drive at `k × ROBOT_SPEED_M_S` inside it. Normal floor has `k = 1`. Look at the maps terrain_sim and terrain_robot.

Step 2: The cost of a step. Your planner takes the input cost_mode. With cost_mode = "distance", the cost of a step is the distance travelled, as in section 2. Add cost_mode = "time", where the cost of a step into cell n' is the time needed to cross it:

`cost(n, n') = d(n, n') / (k(n') · v_max)`

Here `d(n, n')` is the distance between the two cells (0.01 m), k(n') is the traversability of the cell entered, and `v_max = ROBOT_SPEED_M_S`.

Step 3: An admissible heuristic. The heuristic must also be a time. Use the time needed to drive the straight line to the goal at the highest possible speed:

`h(n) = d_straight(n, goal) / v_max`

In your report, explain why this heuristic is admissible.

Step 4: Already added. Because steps no longer all cost the same, a cell can first be reached by an expensive path and later by a cheaper one. Instead of an array of 0s and 1s, keep a 200×200 array with the lowest g found so far for each cell. Only create a neighbor if its g is lower than the stored value.

Step 5: Unit tests. Complete the minimum time unit tests in Part 3 of `lab_4.py`. Use a slow strip across the straight line between start and goal:

- with `k = 0.2`, the "time" planner must go around the strip and the "distance" planner through it;
- with `k = 0.9`, both must go through it.

Compute the time of each path by hand and compare it with `path_time_s(P)`.

4. TRAJECTORY FOLLOWING

Step 1: From path to trajectory. Write `path_to_trajectory(P, cost_mode)`. It turns the path of cells from your planner into a trajectory with one point per cell:

turns the path of cells from your planner into a trajectory with one point per cell:

- **Position**: the centre of the cell, in meters (cell_to_meters).
- **Time**: `t_0 = 0`, then

  ```
  t_j = t_(j−1) + d_j / s_j
  ```

  where `d_j` is the distance from point `j−1` to point `j`, and `s_j` is the speed allowed in cell `j`: `s_j = v_max` for "distance", and `s_j = k_j · v_max` for "time".
- **Heading**: `θ_j` points from point `j` to the point `HEADING_LOOKAHEAD_CELLS` cells further along the path. A single step on the grid always points at 0°, 90°, 180° or −90°, even on a diagonal.

Step 2: The trajectory tracker. Write `trajectory_tracking_controller(T, X, t)`. At each time step t, as in Lecture 06A:

1. find the pose `p_t` on T that has the same time stamp as t;
2. find the pose p_des,t that is a time `Δt = TRAJ_LOOKAHEAD_S` further along T (the last pose of T if the trajectory is over);
3. return the wheel speeds of your point tracking controller with `p_des,t` as the desired pose.

Compare it with your Lab 3 path tracker, which used a distance Δ instead of a time Δt.

Step 3: Unit tests. Complete the trajectory unit tests in Part 4 of `lab_4.py`. Plan from (0, 0) to (1.5, 0) m:

1. on box_sim with "distance";
2. on terrain_sim with "time".

Simulate the robot with `robot_simulator` until it reaches the end of the trajectory. Record the largest distance between the robot and `p_t` (the tracking error), and check that the robot never touches an obstacle.

Step 4: In the GUI. TRACK PLAN uses your trajectory tracker when Track: trajectory is

selected. Select Track: path (Lab 3) to compare with your Lab 3 path tracker. The orange dot shows `p_t`, where the trajectory says the robot should be now. At the end, the GUI shows the time taken, the planned time and the largest tracking error.

5. A* IN SIMULATION

Step 1: Test your A* in simulation for various initial states and desired positions. Make sure:

1. An optimal path is constructed
2. The path tracking works

Create 2 new maps where path planning is more complex and interesting. Screen record the simulations of the robot tracking the path. This will be submitted on BrightSpace. Take screenshots of planned paths. These will be useful for your report.

6. A* ANALYSIS

Run an analysis on the the path length as a function of map density. Specifically, create 5 maps of increasing density (percent of the 200x200 cells that are occupied by obstacles). You may want to write a function that does this, ensuring the map is fully connected.

Call the A* planner 100 times for each map, with randomly selected goals and initial robot positions for each plan call. Record the path length and time taken to construct the plan for each plan call. You may want to write a function that calls the planner and logs the time it took.

Step 1: Plot the mean path length (in grid cell units or meters) as a function of map density.

Step 2: Plot the mean planning time (in seconds) as a function of map density.

Be sure to include these plots in your report.

7. A* ON THE REAL ROBOT

Try running the A* with the real robot instead of simulation. Create 2 new maps for these tests. Note if you add real obstacles, make sure that:

1. You update the map file with the obstacles.
2. You leave enough space for the robot to navigate within.

Video record the trials of the robot tracking the path. This will be submitted on BrightSpace. Simultaneously take screenshots of planned paths and photos of the robot in action. These will be useful for your report

8. REPORT WRITING

For the report you submit, **only the abstract is to be written in the ZeroAI lab**. Work hard to make the abstract read well and provide a concise overview. It should only be one paragraph long. Ask TAs for feedback.

You can write the rest of the report outside of the ZeroAI lab. Start by writing bullet points for each section. Then fill in the gaps. You are allowed to use Gen AI tools to help with writing at this point, but in the end, **quality matters**. Proof read your report. Make sure it adheres to the format described below in the Appendix.

9. QUICK SURVEY

If you have agreed to participate in the research survey on the impact of the Zero AI lab, please fill out the survey linked here: SURVEY LINK

APPENDIX: REPORT DETAILS

The formal report should be of length 3-6 pages, written using IEEE format as mentioned on the website assignment schedule page. For this assignment, be sure to include all plots mentioned above, and more plots if you think they are significant. Assume the reader knows about robotics, but that they are not familiar with our class. Sections should include:

1. **Abstract** - The section should provide an overview of the work. There should be a minimum of one sentence that informs the reader about the motivation, method, experiment, and results. Be sure to provide at least two significant quantifiable results from your results that a reader may find interesting.
2. **Introduction** - Use at least one paragraph to describe the motivation for path

following. Use a paragraph to provide an overview of each section of the paper. E.g. “The method section will describe an overview of the A* planner implement, after which the experimental design section will detail how they were validated.”.

3. **Method** - Describe your A* planner and how it works. Be sure to include equations for your heuristic, and state whether or not it is admissable and why.
4. **Experiment design** - Use images, photos, figures to explain the experimental setup and how physical measurements were taken.
5. **Experimental results** - Show all your plots and images here. Discuss assumptions problems, successes, etc. Label all plots. Each plot should be referred to at least once in your text discussions.
6. **Conclusion** - Present a high level understanding of the performance of your A* planner. Mention efforts that could be taken in the future to improve motion planning.
7. **References** - Cite key references. You may want to do a little research to cite key textbooks, controllers you found.

We will use the following grading rubric for a total of 100 ponts:

- 10 - Abstract 10
- 20 - Method (clear & correct) 20
- 50 - Results 50
- 20 - Spelling/Format/Grammar

Christopher Clark c.clark@nyu.edu